

COLLEGE CODE : 9623
**COLLEGE NAME : AMRITA COLLEGE OF ENGINEERING
AND TECHNOLOGY**
DEPARTMENT : CSE
STUDENT NM-ID : EC5D94E5D6488318FDA4A53CFB075AB6
ROLL NO : 40
DATE : 22/9/2025

COMPLETED THE PROJECT NAMED AS PHASE 3

TECHNOLOGY PROJECT NAME : USER REGISTRATION WITH VALIDATION

SUBMITTED BY,

NAME : H.INDHU KRISHNA
MOBILE NO : 9345827601

User Registration With Validation – MVP Implementation

1. Project Setup

The first step is setting up the development environment and selecting the technology stack.

Requirements: Users should be able to register with valid credentials, log in, and have their information stored securely.

Tech Stack:

Frontend: React.js (form and validation).

Backend: Node.js with Express (API handling).

Database: MongoDB/MySQL (user storage).

Tools: VS Code, Git, Postman, GitHub.

Setup Steps: Install dependencies (React, Express, database connectors, bcrypt for hashing). Initialize Git and connect to GitHub. Define a simple folder structure for frontend, backend, and tests.

2. Core Features Implementation

The MVP includes only the essential features:

Registration Form: Input fields for name, email, password, confirm password.

Validation: Check empty fields, valid email format, unique email, and password strength (e.g., min 8 characters).

Backend API: Handles requests, validates inputs, hashes passwords, and saves users.

Login System: Verifies user credentials and generates authentication tokens (JWT).

Error Handling: Display messages for invalid inputs or duplicate users.

3. Data Storage (Local State / Database)

Local State: Temporarily store form inputs before submission (e.g., React useState). Helps in real-time validation like showing “Password too short.”

Database: Stores user details permanently. Schema example:

User { id, name, email (unique), password (hashed) }

Security: Store only hashed passwords, enforce unique constraints on email, and use validation rules at both frontend and backend.

4. Testing Core Features

Testing ensures the system works before deployment.

Unit Tests: Validate email format, password rules, and API functions.

Integration Tests: Test registration flow from frontend to backend to database.

Manual Testing Checklist:

- ☐ Can new users register successfully?
- ☐ Are duplicate emails rejected?
- ☐ Can users log in with valid credentials?
- ☐ Are invalid logins rejected?

Bug Fixing: Errors found during tests should be logged and resolved immediately.

5. Version Control (GitHub)

Version control helps track progress and manage collaboration.

Setup: Initialize Git, create a GitHub repository, and connect.

Branching: Use main for stable code, dev for active development, and feature-specific branches (e.g., feature/register).

Commit Practice: Commit small changes with clear messages ("Added email validation").

Collaboration: Use pull requests (PRs) for reviews and merge only tested features.

Releases: Tag stable versions (v1.0) for deployment checkpoints.